Heap

Wednesday, May 27, 2026 10:43 AM

A **Heap** is a complete binary tree data structure that satisfies the heap property: in a min-heap, the value of each child is greater than or equal to its parent, and in a max-heap, the value of each child is less than or equal to its parent. Heaps are commonly used to implement priority queues, where the smallest (or largest) element is always at the root.

A Binary Heap is a special type of complete binary tree, meaning all levels are filled except possibly the last, which is filled from left to right.

- It allows fast access to the minimum or maximum element. There are two types of binary heaps: Min Heap and Max Heap.
- **Min Heap**: The value of the root node is the smallest, and this property is true for all subtrees.
- **Max Heap**: The value of the root node is the largest, and this rule also applies to all subtrees.
- Binary heaps are commonly used in priority queues and heap sort algorithms because of their efficient insertion and deletion operations.

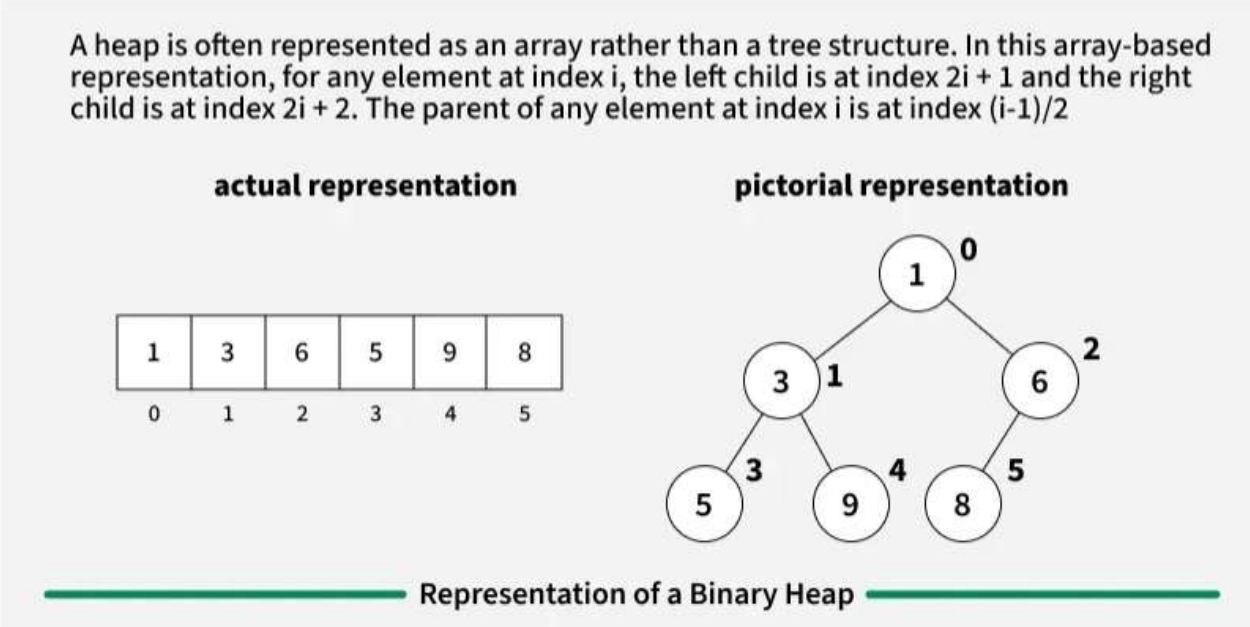
A Binary Heap is a **Complete Binary Tree**. A binary heap is typically represented as an array.

- The root element will be at arr[0].
- The below table shows indices of other nodes for the i^{th} node, i.e., arr[i]:

  arr[(i-1)/2] Returns the parent node

  arr[(2*i)+1] Returns the left child node

  arr[(2*i)+2] Returns the right child node



Properties of Heap:

- In a heap, the minimum or maximum element is always at the root, allowing constant-time access. For a node at index i (0-based indexing), its left child is at index 2i + 1 and its right child is at index 2i + 2.
- Since a heap is a complete binary tree, all levels are fully filled except possibly the last, which is filled from left to right.

- When inserting an element, it is added at the last available position, and then the nodes are rearranged to maintain the heap property.
- When removing an element, the root is swapped with the last node to remove either the maximum or minimum value. The remaining nodes are then rearranged to ensure the heap property is preserved.

Operations Supported by Heap:
Operations supported by **min - heap** and **max - heap** are same. The difference is just that min-heap contains minimum element at root of the tree and max - heap contains maximum element at the root of the tree.
Heapify:
Heapify is the process of rearranging elements to maintain the heap property. It is performed in two main scenarios:

- **When the root is removed:** The root is replaced with the last node, and then heapify is called to restore the heap property.
- **When building a heap:** Heapify is applied from the last internal node up to the root to ensure the entire tree satisfies the heap property.

This operation has a time complexity of **O(log n).**

- In a max-heap, heapify ensures the maximum element is at the root, and all subtrees maintain the same property.
- In a min-heap, it ensures the minimum element is at the root, with all subtrees following the min-heap property.

Insertion:
When a new element is inserted into a heap, it may violate the heap property. To restore the heap structure, a heapify operation is performed, which ensures the heap properties are maintained.
Time complexity **- O(log n).**
Deletion:
Deleting an element from a heap means removing the root node, replacing it with the last node in the heap, and then performing heapify to restore the heap's order.
Time complexity **- O(log n).**
getMax (For max-heap) or getMin (For min-heap):
It finds the maximum element or minimum element for **max-heap** and **min-heap** respectively and as we know minimum and maximum elements will always be the root node itself for min-heap and max-heap respectively.
Time Complexity **- O(1).**

```
# Enables Python 3 print function behavior in Python 2 environments
from __future__ import print_function
import math

class MinHeap:
    def __init__(self):
        # Initialize the heap as an empty list
        self.arr = []

    # Get the index of the left child
    def left(self, i): return 2 * i + 1

    # Get the index of the right child
    def right(self, i): return 2 * i + 2

    # Get the index of the parent
    def parent(self, i): return (i - 1) // 2

    # Return the minimum element without removing it
    def get_min(self):
        return self.arr[0] if self.arr else None
```

```
    # Insert a new key into the heap
    def insert(self, k):
        self.arr.append(k)
        i = len(self.arr) - 1

        # Fix the min heap property by bubbling up
        while i > 0 and self.arr[self.parent(i)] > self.arr[i]:
            p = self.parent(i)
            self.arr[i], self.arr[p] = self.arr[p], self.arr[i]
            i = p

    # Decrease the value of a key at a specific index
    def decrease_key(self, i, new_val):
        self.arr[i] = new_val

        # Percolate the new smaller value up to maintain heap property
        while i != 0 and self.arr[self.parent(i)] > self.arr[i]:
            p = self.parent(i)
            self.arr[i], self.arr[p] = self.arr[p], self.arr[i]
            i = p

    # Remove and return the root (minimum) element
    def extract_min(self):
        if len(self.arr) <= 0: return None
        if len(self.arr) == 1: return self.arr.pop()

        res = self.arr[0]
        # Replace root with the last element and heapify down
        self.arr[0] = self.arr.pop()
        self.min_heapify(0)
        return res

    # Delete a key at index i by forcing it to root and extracting
    def delete_key(self, i):

        # Use negative infinity to ensure it becomes the new root
        self.decrease_key(i, -float('inf'))

        # Remove the forced root
        self.extract_min()

    # Recursive method to fix the heap property downwards
    def min_heapify(self, i):
        l, r, n = self.left(i), self.right(i), len(self.arr)
        smallest = i

        # Find the smallest among root, left child, and right child
        if l < n and self.arr[l] < self.arr[smallest]: smallest = l
        if r < n and self.arr[r] < self.arr[smallest]: smallest = r

        # If the root is not the smallest, swap and continue heapifying
        if smallest != i:
            self.arr[i], self.arr[smallest] = self.arr[smallest], self.arr[i]
            self.min_heapify(smallest)

# --- Execution ---
h = MinHeap()
```

```
h.insert(3)
h.insert(2)
h.delete_key(1)
h.insert(15)
h.insert(5)
h.insert(4)
h.insert(45)

# Prints values on the same line separated by spaces
print(h.extract_min(), end=" ")
print(h.get_min(), end=" ")

h.decrease_key(2, 1)
print(h.extract_min())
```

Heap Sort

Step 1: Treat the Array as a Complete Binary Tree

We first need to visualize the array as a complete binary tree. For an array of size n, the root is at index 0, the left child of an element at index i is at 2i + 1, and the right child is at 2i + 2.

Step 2: Build a Max Heap

Step 3: Sort the array by placing largest element at end of unsorted array.

```
# To heapify a subtree rooted with node i
def heapify(arr, n, i):

    # Initialize largest as root
    largest = i

    # left index = 2*i + 1
    l = 2 * i + 1

    # right index = 2*i + 2
    r = 2 * i + 2

    # If left child is larger than root
    if l < n and arr[l] > arr[largest]:
        largest = l

    # If right child is larger than largest so far
    if r < n and arr[r] > arr[largest]:
        largest = r

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Recursively heapify the affected sub-tree
        heapify(arr, n, largest)

# Main function to do heap sort
def heapSort(arr):
    n = len(arr)

    # Build heap (rearrange vector)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
```

```
    # One by one extract an element from heap
    for i in range(n - 1, 0, -1):

        # Move current root to end
        arr[0], arr[i] = arr[i], arr[0]

        # Call max heapify on the reduced heap
        heapify(arr, i, 0)

if __name__ == "__main__":
    arr = [9, 4, 3, 8, 10, 2, 5]

    heapSort(arr)

    for i in range(len(arr)):
        print(arr[i], end=" ")
```

Time Complexity: O(n log n)
Auxiliary Space: O(log n), due to the recursive call stack. However, auxiliary space can be O(1) for iterative implementation.

```
# To heapify a subtree rooted with node i
def heapify(arr, n, i):

    # Initialize largest as root
    largest = i

    # left index = 2*i + 1
    l = 2 * i + 1

    # right index = 2*i + 2
    r = 2 * i + 2

    # If left child is larger than root
    if l < n and arr[l] > arr[largest]:
        largest = l

    # If right child is larger than largest so far
    if r < n and arr[r] > arr[largest]:
        largest = r

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Recursively heapify the affected sub-tree
        heapify(arr, n, largest)

# Main function to do heap sort
def heapSort(arr):
    n = len(arr)

    # Build heap (rearrange vector)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract an element from heap
```

```
    for i in range(n - 1, 0, -1):

        # Move current root to end
        arr[0], arr[i] = arr[i], arr[0]

        # Call max heapify on the reduced heap
        heapify(arr, i, 0)

if __name__ == "__main__":
    arr = [9, 4, 3, 8, 10, 2, 5]

    heapSort(arr)

    for i in range(len(arr)):
        print(arr[i], end=" ")
```

Heap Data Structure is generally taught with Heapsort. Heapsort algorithm has limited uses because Quicksort is better in practice. Nevertheless, the Heap data structure itself is enormously used.

1. **Priority Queues**: Heaps are commonly used to implement priority queues, where elements with higher priority are extracted first. This is useful in many applications such as scheduling tasks, handling interruptions, and processing events.
2. **Sorting Algorithms**: Heapsort, a comparison-based sorting algorithm, is implemented using the Heap data structure. It has a time complexity of O(n log n), making it efficient for large datasets.
3. **Graph algorithms**: Heaps are used in graph algorithms such as Prim's Algorithm, Dijkstra's algorithm., and the A* search algorithm.
4. **Lossless Compression**: Heaps are used in data compression algorithms such as Huffman coding, which uses a priority queue implemented as a min-heap to build a Huffman tree.
5. **Medical Applications**: In medical applications, heaps are used to store and manage patient information based on priority, such as vital signs, treatments, and test results.
6. **Load balancing**: Heaps are used in load balancing algorithms to distribute tasks or requests to servers, by processing elements with the lowest load first.
7. ***Order statistics:*** The Heap data structure can be used to efficiently find the kth smallest (or largest) element in an array. See method 4 and 6 of this post for details.
8. **Resource allocation:** Heaps can be used to efficiently allocate resources in a system, such as memory blocks or CPU time, by assigning a priority to each resource and processing requests in order of priority.
9. **Job scheduling**: The heap data structure is used in job scheduling algorithms, where tasks are scheduled based on their priority or deadline. The heap data structure allows efficient access to the highest-priority task, making it a useful data structure for job scheduling applications.

Building Heap from Array

To build a Max Heap from an array, treat the array as a complete binary tree and heapify nodes from the last non-leaf node up to the root in reverse level order. Leaf nodes already satisfy the heap property, so we start from the last non-leaf node, and for each subtree, we compare the parent with its children. Whenever a child node is greater than the parent, we swap them and continue heapifying that subtree to ensure the Max Heap property is maintained throughout.

Note :

- Root is at index 0.
- Left child of node i -> 2*i + 1.
- Right child of node i -> 2*i + 2.
- Parent of node i -> (i-1)/2.

- Last non-leaf node -> parent of last node -> (n/2) - 1.

```
# To heapify a subtree
def heapify(arr, n, i):

    # Initialize largest as root
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2

    # If left child is larger than root
    if l < n and arr[l] > arr[largest]:
        largest = l

    # If right child is larger than largest so far
    if r < n and arr[r] > arr[largest]:
        largest = r

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Recursively heapify the affected sub-tree
        heapify(arr, n, largest)

# Function to build a Max-Heap from the given array
def buildHeap(arr):
    n = len(arr)

    # Index of last non-leaf node
    startIdx = (n // 2) - 1

    # Perform reverse level order traversal
    # from last non-leaf node and heapify
    # each node
    for i in range(startIdx, -1, -1):
        heapify(arr, n, i)

if __name__ == "__main__":
    # Binary Tree Representation
    # of input array
    #              1
    #           /    \
    #         3       5
    #       /  \     /  \
    #     4     6  13  10
    #    / \   / \
    #   9   8  15 17
    arr = [1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17]
    n = len(arr)

    # Function call
    buildHeap(arr)

    for i in range(n):
        print(arr[i], end=" ")
    print()

    # Final Heap:
```

```
#           17
#          /   \
#         15     13
#        /  \    / \
#       9    6  5   10
#      / \   / \
#     4   8 3   1
```

Operations on Min Heap

1. getMin(): It returns the root element of Min Heap. Time Complexity of this operation is O(1).
2. extractMin(): Removes the minimum element from MinHeap. Time Complexity of this Operation is O(Log n) as this operation needs to maintain the heap property (by calling heapify()) after removing root.
3. insert(): Inserting a new key takes O(Log n) time. We add a new key at the end of the tree. If new key is larger than its parent, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.

```
class MinHeap:
    def __init__(self):
        self.a = []

    """Insert a new element into the Min Heap."""
    def insert(self, val):
        self.a.append(val)
        i = len(self.a) - 1
        while i > 0 and self.a[(i - 1) // 2] > self.a[i]:
            self.a[i], self.a[(i - 1) // 2] = self.a[(i - 1) // 2], self.a[i]
            i = (i - 1) // 2

    """Delete a specific element from the Min Heap."""
    def delete(self, value):
        i = -1
        for j in range(len(self.a)):
            if self.a[j] == value:
                i = j
                break
        if i == -1:
            return

        self.a[i] = self.a[-1]
        self.a.pop()

        parent = (i - 1) // 2

        if i > 0 and self.a[i] < self.a[parent]:
            while i > 0 and self.a[(i - 1)//2] > self.a[i]:
                self.a[i], self.a[(i - 1)//2] = self.a[(i - 1)//2], self.a[i]
                i = (i - 1)//2
        else:
            self.minHeapify(i, len(self.a))

    """Heapify function to maintain the heap property."""
    def minHeapify(self, i, n):
        smallest = i
        left = 2 * i + 1
        right = 2 * i + 2

        if left < n and self.a[left] < self.a[smallest]:
```

```
            smallest = left
        if right < n and self.a[right] < self.a[smallest]:
            smallest = right
        if smallest != i:
            self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
            self.minHeapify(smallest, n)

    """Search for an element in the Min Heap."""
    def search(self, element):
        for j in self.a:
            if j == element:
                return True
        return False

    def getMin(self):
        return self.a[0] if self.a else None

    def printHeap(self):
        print("Min Heap:", self.a)

# Example Usage
if __name__ == "__main__":
    h = MinHeap()
    values = [10, 7, 11, 5, 4, 13]
    for value in values:
        h.insert(value)
    h.printHeap()

    h.delete(7)
    print("Heap after deleting 7:", h.a)

    print("Searching for 10 in heap:", "Found" if h.search(10) else "Not Found")
    print("Minimum element in heap:", h.getMin())
from heapq import heapify, heappush, heappop
heap = []
heapify(heap)

heappush(heap, 10)
heappush(heap, 30)
heappush(heap, 20)
heappush(heap, 400)

print("Head value of heap : "+str(heap[0]))

print("The heap elements : ")
for i in heap:
    print(i, end = ' ')
print("\n")

element = heappop(heap)

print("The heap elements : ")
for i in heap:
    print(i, end = ' ')

from queue import PriorityQueue
q = PriorityQueue()
```

```
q.put(10)
q.put(20)
q.put(5)

print(q.get())
print(q.get())

print('Items in queue :', q.qsize())
print('Is queue empty :', q.empty())
print('Is queue full :', q.full())
```

Operations on Max Heap:

1. **getMax()**: It returns the root element of Max Heap. Time Complexity of this operation is **O(1)**.
2. **extractMax()**: Removes the maximum element from MaxHeap. Time Complexity of this Operation is **O(log n)** as this operation needs to maintain the heap property (by calling heapify()) after removing the root.
3. **insert()**: Inserting a new key takes **O(log n)** time. We add a new key at the end of the tree. If the new key is smaller than its parent, then we don&#x2019t need to do anything. Otherwise, we need to traverse up to fix the violated heap property.

```
import sys

class MaxHeap:
    def __init__(self, cap):
        self.cap = cap
        self.n = 0
        self.a = [0] * (cap + 1)
        self.a[0] = sys.maxsize
        self.root = 1

    def parent(self, i):
        return i // 2

    def left(self, i):
        return 2 * i

    def right(self, i):
        return 2 * i + 1

    def isLeaf(self, i):
        return i > (self.n // 2) and i <= self.n

    def swap(self, i, j):
        self.a[i], self.a[j] = self.a[j], self.a[i]

    def maxHeapify(self, i):
        if not self.isLeaf(i):
            largest = i
            if self.left(i) <= self.n and self.a[i] < self.a[self.left(i)]:
                largest = self.left(i)
            if self.right(i) <= self.n and self.a[largest] < self.a[self.right(i)]:
                largest = self.right(i)
            if largest != i:
                self.swap(i, largest)
                self.maxHeapify(largest)

    def insert(self, val):
        if self.n >= self.cap:
```

```
            return
        self.n += 1
        self.a[self.n] = val
        i = self.n
        while self.a[i] > self.a[self.parent(i)]:
            self.swap(i, self.parent(i))
            i = self.parent(i)

    def extractMax(self):
        if self.n == 0:
            return None
        max_val = self.a[self.root]
        self.a[self.root] = self.a[self.n]
        self.n -= 1
        self.maxHeapify(self.root)
        return max_val

    def printHeap(self):
        for i in range(1, (self.n // 2) + 1):
            print(f"PARENT: {self.a[i]}", end=" ")
            if self.left(i) <= self.n:
                print(f"LEFT: {self.a[self.left(i)]}", end=" ")
            if self.right(i) <= self.n:
                print(f"RIGHT: {self.a[self.right(i)]}", end=" ")
            print()

# Example
if __name__ == "__main__":
    print("The maxHeap is:")
    h = MaxHeap(15)
    vals = [5, 3, 17, 10, 84, 19, 6, 22, 9]
    for val in vals:
        h.insert(val)

    h.printHeap()
    print("The Max val is", h.extractMax())
# Python3 program to demonstrate heapq (Max Heap)

from heapq import heappop, heappush, heapify

# Create an empty heap
h = []
heapify(h)

# Add elements (multiplying by -1 to simulate Max Heap)
heappush(h, -10)
heappush(h, -30)
heappush(h, -20)
heappush(h, -400)

# Print max element
print("Max:", -h[0])

# Print heap elements
print("Heap:", [-i for i in h])

# Pop max element
heappop(h)
```

```
# Print heap after removal
print("Heap after pop:", [-i for i in h])

from queue import PriorityQueue

q = PriorityQueue()

# Insert elements into the queue (negate values to simulate Max Heap)
q.put(-10)
q.put(-20)
q.put(-5)

# Remove and return the highest priority item (convert back to positive)
print(-q.get())  # 20 (highest value)
print(-q.get())  # 10

# Check queue size
print('Items in queue:', q.qsize())

# Check if queue is empty
print('Is queue empty:', q.empty())

# Check if queue is full
print('Is queue full:', q.full())
```

A heap queue (also called a priority queue) is a data structure that allows quick access to the smallest (min-heap) or largest (max-heap) element.

- By default, heaps are implemented as min-heaps.
- Smallest element is always at the root and largest element is located among the leaf nodes of the heap.
- Python provides a built-in module called heapq that allows to create and work with heap queues

```
import heapq
li = [25, 20, 15, 30, 40]
heapq.heapify(li)
print("Heap queue:", li)
```

Operation	Function	Description
Create	heapq.heapify()	Converts a regular list into a valid min-heap
Push	heapq.heappush()	Adds a new element to the heap while maintaining the heap property
Pop	heapq.heappop()	Removes and returns the smallest element from the heap
Peek	heap[0]	Accesses the smallest element without removing it
Push and Pop	heapq.heappushpop()	Pushes a new element and removes the smallest element in one step
Replace	heapq.heapreplace()	Removes the smallest element and inserts a new element in one operation

By default, Python's heapq implements a min-heap. To create a max-heap simply invert the values (store negative numbers).

```
import heapq
nums = [10, 20, 15, 30, 40]

# Convert into a max-heap by inverting values
max_heap = [-n for n in nums]
heapq.heapify(max_heap)

# Access largest element (invert sign again)
print("Largest element:", -max_heap[0])
```

Appending and Popping Elements

Elements can be inserted and removed while maintaining the heap property:

- heapq.heappush(heap, item) adds a new element to the heap.
- heapq.heappop(heap) removes and returns the smallest element.

```
import heapq
h = [10, 20, 15, 30, 40]
heapq.heapify(h)

# Appending an element
heapq.heappush(h, 5)
print(h)

# Pop the smallest element from the heap
min = heapq.heappop(h)
print("Smallest:", min)
print(h)
```

Appending and Popping Simultaneously

heapq.heappushpop() pushes a new element onto the heap and removes the smallest element in a single operation.

```
import heapq
h = [10, 20, 15, 30, 40]
heapq.heapify(h)
min = heapq.heappushpop(h, 5)
print(min)
print(h)
```

Finding Largest and Smallest Elements

heapq.nlargest() and heapq.nsmallest() return the required number of largest or smallest elements from an iterable.

```
import heapq
h = [10, 20, 15, 30, 40]
heapq.heapify(h)

maxi = heapq.nlargest(3, h)
print("3 largest elements:", maxi)

min = heapq.nsmallest(3, h)
print("3 smallest elements:", min)
```

Replace and Merge Operations

heapq.heapreplace() removes the smallest element and inserts a new one in a single step, returning the removed value. While, heapq.merge() combines multiple sorted iterables into a single sorted sequence.

```
import heapq
h1 = [10, 20, 15, 30, 40]
heapq.heapify(h1)

min = heapq.heapreplace(h1, 5)
print(min)
print(h1)

h2 = [2, 4, 6, 8]
h3 = list(heapq.merge(sorted(h1), sorted(h2)))
print("Merged heap:", h3)
```

Advantages	Disadvantages
Fast for insertion and removal with priority.	Not suitable for complex data manipulations
Uses less memory than some other data types.	No direct access to middle items.
Simple to use with the heapq module.	Can't fully sort the items automatically.
Works in many cases like heaps and priority queues.	Not safe with multiple threads at the same time.

The heapq.heapify() function in Python is used to transform a regular list into a valid min-heap. A min-heap is a binary tree where the smallest element is always at the root. This method is highly useful when you need to create a heap structure from an unordered list and maintain the heap property efficiently.

How Does heapq.heapify() Work?

- The heapq.heapify() function rearranges the elements in the list to make it a valid min-heap.
- The heap property is maintained after this operation, so the smallest element will always be at index 0.
- It runs in O(n) time complexity, where n is the number of elements in the list. This is more efficient than using heapq.heappush() repeatedly to insert elements, which would take O(n log n) time.

Introduction to Priority Queue

A **priority queue** is a type of queue where each element is associated with a priority value, and elements are served based on their priority rather than their insertion order.

- Elements with higher priority are retrieved or removed before those with lower priority.
- When a new item is added, it is inserted according to its priority.

The **binary heap** is the most common implementation of a priority queue:

- A **min-heap** allows quick access to the element with the smallest value.
- A **max-heap** allows quick access to the element with the largest value.
- Binary heaps are complete binary trees, making them easy to implement using arrays.
- Using arrays provides cache-friendly memory access, improving performance.

Priority queues are widely used in algorithms such as Dijkstra's shortest path algorithm, Prim's minimum spanning tree algorithm, and Huffman coding for data compression.

- **Min-Heap:** In this queue, elements with lower values have higher priority. For example, with elements 4, 6, 8, 9, and 10, 4 will be dequeued first since it has the smallest value, and the dequeue operation will return 4.
- **Max-Heap:** Elements with higher values have higher priority. The root of the heap is the highest element, and it is dequeued first. The queue adjusts by maintaining the heap property after each insertion or deletion.

A Binary Heap is ideal for priority queue implementation as it offers better performance The largest key is at the top and can be removed in O(log n) time, with the heap property restored efficiently. If a new entry is inserted immediately, its O(log n) insertion time may overlap with restoration. Since heaps use contiguous storage, they efficiently handle large datasets while ensuring logarithmic time for insertions and deletions.

- **insert(p):** Inserts a new element with priority p.
- **pop():** Extracts an element with maximum priority.
- **getMax():** Returns an element with maximum priority.

```
# Returns index of parent
def parent(i):
    return (i - 1) // 2

# Returns index of left child
def leftChild(i):
    return 2 * i + 1

# Returns index of right child
def rightChild(i):
    return 2 * i + 2

# Shift up to maintain max-heap property
def shiftUp(i, arr):
    while i > 0 and arr[parent(i)] < arr[i]:
        arr[parent(i)], arr[i] = arr[i], arr[parent(i)]
        i = parent(i)

# Shift down to maintain max-heap property
def shiftDown(i, arr, size):
```

```
    maxIndex = i
    l = leftChild(i)
    if l < size and arr[l] > arr[maxIndex]:
        maxIndex = l
    r = rightChild(i)
    if r < size and arr[r] > arr[maxIndex]:
        maxIndex = r

    if i != maxIndex:
        arr[i], arr[maxIndex] = arr[maxIndex], arr[i]
        shiftDown(maxIndex, arr, size)

# Insert a new element
def insert(p, arr):
    arr.append(p)
    shiftUp(len(arr) - 1, arr)

# Extract element with maximum priority
def pop(arr):
    size = len(arr)
    if size == 0:
        return -1
    result = arr[0]
    arr[0] = arr[size - 1]
    arr.pop()
    shiftDown(0, arr, len(arr))
    return result

# Get current maximum element
def getMax(arr):
    if not arr:
        return -1
    return arr[0]


# Print heap
def printHeap(arr):
    print(" ".join(map(str, arr)))

# Driver code
if __name__ == "__main__":
    pq = []

    insert(45, pq)
    insert(20, pq)
    insert(14, pq)
    insert(12, pq)
    insert(31, pq)
    insert(7, pq)
    insert(11, pq)
    insert(13, pq)
    insert(7, pq)

    print("Priority Queue after inserts: ", end="")
    printHeap(pq)

    # Demonstrate getMax
    print("Current max element:", getMax(pq))
```

```
    # Demonstrate extractMax
    pop(pq)
    print("Priority Queue after extracting max: ", end="")
    printHeap(pq)
```

Time Complexity: O(log n), for all the operation, except getMax(), which has time complexity of O(1).
Space Complexity: O(n)

Difference between Priority Queue and Queue

There is no priority attached to elements in a queue, the rule of first-in-first-out(FIFO) is implemented whereas, in a priority queue, the elements have a priority. The elements with higher priority are served first.

Operations on a Priority Queue

A typical priority queue supports the following operations:

1) Insertion (push) : When a new item is inserted in a Max-Heap, if it has the highest value, it becomes the root; in a Min-Heap, if it has the smallest value, it becomes the root. Otherwise, it is placed so that all higher-priority elements (larger in Max-Heap, smaller in Min-Heap) are accessed before it.

2) Deletion(pop) : We usually remove the highest-priority item, which is always at the top. After removing it, the next highest-priority item automatically takes its place at the top.

3) top : This operation only returns the highest priority item (which is typically available at the top) and does not make any change to the priority queue.

```
import heapq

if __name__ == "__main__":
    # Create a max priority queue using min heap with negative values
    pq = []

    # Insert elements into the priority queue
    heapq.heappush(pq, -10)
    heapq.heappush(pq, -30)

    print("Priority Queue elements (max-heap):")

    # Access elements and pop them one by one
    while pq:

        # Access the top (highest priority element)
        print("Top element:", -pq[0])

        # Remove the top element
        heapq.heappop(pq)

        # Print remaining size
        print("Remaining size:", len(pq), "\n")

    # Check if empty
    if not pq:
        print("Priority Queue is now empty.")
```

Merge Two Binary Max Heaps

[Naive Approach] Merge using extra Max Heap - O((N + M)*log(N + M)) Time and O(N + M) Space:

The property of Max Heap is that the top of any Max Heap is always the largest element. We will use this property to merge the given binary Max Heaps.

To do this:

- Create another Max Heap, and insert the elements of given binary max heaps into this one by one.
    - Now repeat above step until all the elements from both given Max Heaps are merged into the third one, ensuring the property of largest element on top (max heapify).
- The third Max Heap will be the required merged Binary Max Heap.

```
import heapq

def merge_heaps(a, b):
    # Create a max heap by negating the elements
    max_heap = []

    # Insert all elements of first list into the max heap
    for num in a:
        heapq.heappush(max_heap, -num)

    # Insert all elements of second list into the max heap
    for num in b:
        heapq.heappush(max_heap, -num)

    merged = []
    # Extract elements from the max heap
    while max_heap:
        merged.append(-heapq.heappop(max_heap))

    return merged

# Sample Input
a = [10, 5, 6, 2]
b = [12, 7, 9]

merged = merge_heaps(a, b)

for num in merged:
    print(num, end=' ')
```

Time Complexity: O((N + M)*log(N + M)), where N is the size of a[] and M is the size of b[].

- Since we are fetching one element after the other from given max heaps, the time for this will be O(N + M)
- Now for each element, we are doing max heapify on the third heap. This heapify operation for element of the heap will take O(log N+M) each time
- Therefore the resultant complexity will be O(N+M)*(log (N+M))

Auxiliary Space: O(N + M), which is the size of the resultant merged binary max heap

[Expected Approach] Merge and then Build Heap - O(N + M) Time and O(N + M) Space:

Another approach is that instead of creating a max heap from the start, flatten the given max heap into arrays and merge them. Then max heapify this merged array. The benefit of doing so, is that, instead of taking O(log N+M) time everytime to apply heapify operation as shown in above approach, this approach will take only O(N+M) time once to build the final heap.

```
# Python3 program to merge two Max heaps

# Standard heapify function to heapify a subtree
# rooted under idx. It assumes that subtrees of node
# are already heapified
def maxHeapify(arr, n, idx):
    if idx >= n:
```

```
        return

    l = 2 * idx + 1
    r = 2 * idx + 2
    max_idx = idx

    if l < n and arr[l] > arr[max_idx]:
        max_idx = l
    if r < n and arr[r] > arr[max_idx]:
        max_idx = r

    if max_idx != idx:
        arr[max_idx], arr[idx] = arr[idx], arr[max_idx]
        maxHeapify(arr, n, max_idx)

# Builds a max heap of given arr[0..n-1]
def buildMaxHeap(arr, n):
    for i in range(n // 2 - 1, -1, -1):
        maxHeapify(arr, n, i)


# Merges max heaps a[] and b[] into merged[]
def mergeHeaps(a, b, n, m):
    merged = a + b
    l = len(merged)
    buildMaxHeap(merged, l)
    return merged


# Sample Input
a = [10, 5, 6, 2]
b = [12, 7, 9]

# Function call
merged = mergeHeaps(a, b, len(a), len(b))

for x in merged:
        print(x, end = " ")
```

Time Complexity: O(N + M)
Auxiliary Space: O(N + M)
Time Complexity: O(N + M), where N is the size of a[] and M is the size of b[].

- Time taken to flatten first binary heap is O(N), which is already provided as input
- Time taken to flatten second binary heap is O(M), which is already provided as input
- Time for merging two arrays = O(N+M)
- Time to build max heap from merged array = O(N+M)
- Therefore the resultant complexity will be O(N+M)

Auxiliary Space: O(N + M), which is the size of the resultant merged binary max heap